News Credibility Monitor

An AI-Powered Multi-Agent System for Automated News Verification

Course: Generative AI Capstone Project

Institution: Newton School of Technology

Date: April 2026

Abstract

This report presents the design, implementation, and deployment of the **News Credibility Monitor**—an AI-powered system that assesses the credibility of news articles through a multi-layered pipeline combining classical machine learning, Retrieval-Augmented Generation (RAG), and multi-agent reasoning. The system employs a TF-IDF vectorizer paired with Logistic Regression for an initial classification signal, an in-memory vector store loaded from precomputed embeddings for evidence retrieval, and three specialised LLM-based reasoning agents whose outputs are synthesised by a final judge agent. The complete pipeline is orchestrated as a LangGraph StateGraph with conditional branching based on ML confidence. The frontend is a React application deployed on Vercel, and the backend is a FastAPI service deployed on Render. Evaluation on the ISOT Fake News Dataset yields 98.88% classification accuracy on the ML layer, while the agentic pipeline provides interpretable, multi-perspective reasoning for each verdict.

Contents

1 Introduction **3**

2 System Overview **3**
2.1 High-Level Architecture . . . 3
2.2 Data Flow Summary . . . 4

3 Methodology **6**
3.1 Text Preprocessing . . . 6
3.2 Machine Learning Classification Layer . . . 6
3.2.1 Feature Extraction . . . 6
3.2.2 Classifier . . . 6
3.2.3 Training Data . . . 7
3.2.4 Output . . . 7
3.3 Retrieval-Augmented Generation (RAG) Layer . . . 7
3.3.1 Embedding Strategy . . . 7
3.3.2 Query-Time Retrieval . . . 7
3.3.3 Conditional RAG . . . 8
3.4 Multi-Agent Reasoning System . . . 8
3.4.1 Agent Topology . . . 8
3.4.2 Agent Personas . . . 8
3.5 Final Judge Agent . . . 9
3.6 Output Assembly and Risk Detection . . . 9
3.7 Agent Workflow . . . 10
3.8 Prompting Strategy . . . 10
3.9 State Management . . . 11

4 System Architecture **12**
4.1 Technology Stack . . . 12

5 Results and Analysis **12**
5.1 ML Model Performance . . . 12
5.2 Conditional RAG Behaviour . . . 13
5.3 Multi-Agent Agreement Patterns . . . 14

6 Discussion **14**
6.1 Strengths . . . 14
6.2 Limitations . . . 14

7 Responsible AI Considerations **15**

8 Conclusion **16**

A Frontend Screenshots **17**

1 Introduction

The proliferation of misinformation in digital media has created an urgent need for automated tools that can assess the credibility of news content at scale. Traditional approaches—either purely statistical classifiers or human fact-checking—each have significant limitations: classifiers produce opaque binary labels with no reasoning, while human review cannot keep pace with the volume of online content.

This project addresses both shortcomings by designing a **hybrid intelligence pipeline** that combines three complementary techniques:

1. **Classical Machine Learning** — A TF-IDF + Logistic Regression classifier provides a fast, high-accuracy initial signal (prediction and confidence score).
2. **Retrieval-Augmented Generation (RAG)** — Semantically similar articles from a verified dataset are retrieved to ground the analysis in evidence, reducing hallucination risk.
3. **Multi-Agent Reasoning** — Three LLM-based agents with distinct analytical perspectives (conservative, sceptical, neutral) independently analyse the article, and a final judge agent synthesises their outputs into a transparent, explainable verdict.

The pipeline is orchestrated using **LangGraph**, a state-machine framework for composable AI workflows, with a conditional branch that bypasses the RAG retrieval step when the ML model's confidence exceeds a configurable threshold (85%). This design balances latency and thoroughness: high-confidence predictions are fast-tracked, while ambiguous cases receive additional evidence before reasoning begins.

The system is fully deployed with a React frontend on Vercel and a FastAPI backend on Render, making it accessible as a web application for demonstration and evaluation.

2 System Overview

The News Credibility Monitor is structured as a two-tier web application with a clear separation between the presentation layer and the analysis engine.

2.1 High-Level Architecture

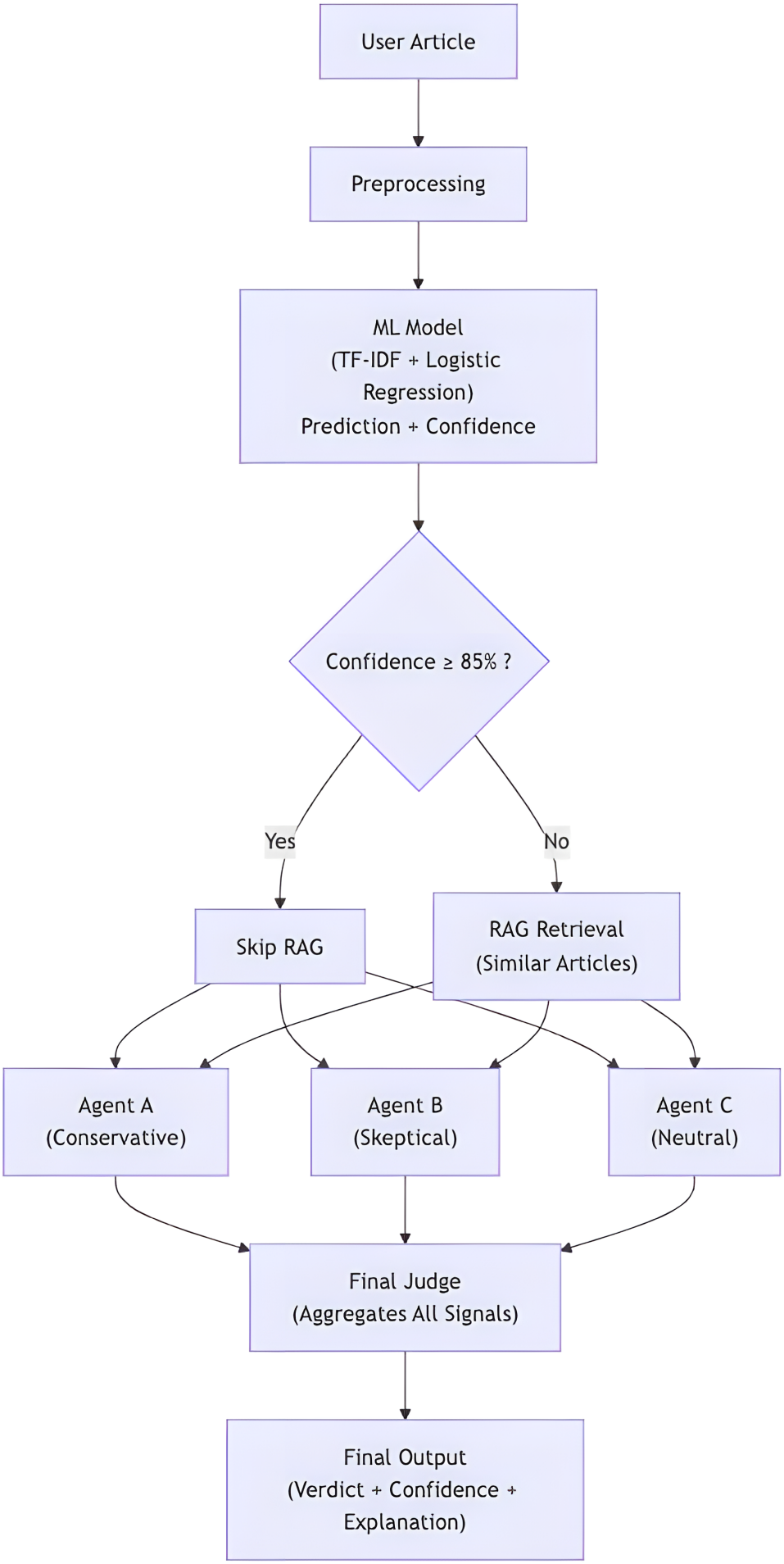
- **Frontend** — A single-page React application (Vite, Tailwind CSS 4, React 19) deployed on Vercel. It collects the user's article text, displays an animated processing pipeline during analysis, and renders the structured verdict with agent reasoning, evidence summaries, and risk indicators.

- **Backend** — A FastAPI service deployed on Render's free tier. It exposes two endpoints: a health-check (`GET /`) and the analysis endpoint (`POST /analyze`). All ML inference, RAG retrieval, and LLM orchestration occur server-side.
- **Keep-Alive** — A GitHub Actions cron job pings the Render service every 14 minutes to prevent the free-tier instance from entering a sleep state.

2.2 Data Flow Summary

The end-to-end data flow for a single analysis request proceeds as follows:

1. User pastes an article ($\geq$50 words) into the web interface.
2. The frontend sends a `POST /analyze` request to the backend.
3. The backend preprocesses the text and runs the LangGraph pipeline.
4. The structured result (verdict, confidence, agent reasoning, evidence, risk factors) is returned as JSON.
5. The frontend normalises the response and renders results with staggered reveal animations.


User Article
Preprocessing
ML Model
(TF-IDF + Logistic
Regression)
Prediction + Confidence
Confidence ≥ 85% ?
Yes
No
Skip RAG
RAG Retrieval
(Similar Articles)
Agent A
(Conservative)
Agent B
(Skeptical)
Agent C
(Neutral)
Final Judge
(Aggregates All Signals)
Final Output
(Verdict + Confidence +
Explanation)

3 Methodology

3.1 Text Preprocessing

All article text passes through a shared cleaning pipeline before entering either the ML or RAG pathways:

1. **Dateline stripping** — Publisher attribution patterns (e.g., `WASHINGTON (Reuters) --`) are removed from the first 50 characters only, preventing accidental content loss on user-supplied text.
2. **Lowercasing** and removal of non-alphabetic characters.
3. **Stopword removal** using NLTK's English stopword list.

This conservative cleaning strategy preserves the semantic content of user-provided articles while normalising the surface form for both the TF-IDF vectorizer and the RAG retrieval query.

3.2 Machine Learning Classification Layer

The ML layer provides the system's initial, low-latency credibility signal.

3.2.1 Feature Extraction

Articles are transformed into numerical feature vectors using a **TF-IDF (Term Frequency–Inverse Document Frequency)** vectorizer with the following configuration:

- `max_features = 10,000`
- `ngram_range = (1, 2)` — captures both unigrams and bigrams

Stopword removal is delegated to the upstream text cleaner to avoid double-filtering.

3.2.2 Classifier

The classifier is a **Logistic Regression** model with the following hyperparameters:

- `max_iter = 1000`
- `solver = lbfgs`
- `C = 1.0` (default regularisation)
- `class_weight = balanced` — compensates for any Real/Fake class imbalance in the training data

3.2.3 Training Data

The model is trained on the **ISOT Fake News Dataset**, which comprises approximately 44,000 articles split between verified real news (sourced from Reuters) and fake news (sourced from unreliable websites flagged by Politifact). The dataset is split 80/20 for training and evaluation (`random_state=42`).

3.2.4 Output

The ML node produces two values that propagate through the rest of the pipeline:

- **Prediction**: `REAL` or `FAKE`
- **Confidence**: The classifier's predicted probability (0–100%), used for conditional routing

3.3 Retrieval-Augmented Generation (RAG) Layer

The RAG layer enriches the analysis with evidence from a verified reference corpus, reducing the LLM agents' tendency to hallucinate.

3.3.1 Embedding Strategy

Embeddings are **precomputed offline** using the `all-MiniLM-L6-v2` model from SentenceTransformers. A stratified sample of 5,000 articles from the ISOT dataset is embedded and serialised as a pickle file (`embeddings.pkl`). This design decision has two key advantages:

1. **No PyTorch at runtime** — The production server does not need the 2 GB+ PyTorch/SentenceTransformers stack, keeping memory usage within Render's 512 MiB free-tier limit.
2. **No persistent disk** — Embeddings are loaded into an **ephemeral ChromaDB client** (in-memory) at startup, eliminating dependency on persistent storage.

3.3.2 Query-Time Retrieval

At query time, the article text is cleaned and passed to ChromaDB's built-in **DefaultEmbeddingFunction** (ONNX-based, lightweight), which generates the query embedding without loading PyTorch. The top-k (k=5) most similar articles are retrieved along with their metadata (label, subject, source) and distance scores.

3.3.3 Conditional RAG

The RAG retrieval step is **conditionally executed** based on the ML model's confidence score:

$$\text{next_node} = \begin{cases} \texttt{rag_node} & \text{if confidence} < 85\% \\ \texttt{agent_a_node} & \text{if confidence} \geq 85\% \end{cases}$$

When the ML model is highly confident, the system bypasses RAG to reduce latency. When confidence is low, retrieved evidence provides critical context for the reasoning agents.

3.4 Multi-Agent Reasoning System

The multi-agent layer is the core differentiator of this system. Rather than relying on a single LLM call, three agents with distinct analytical personas independently evaluate the article, and a judge agent aggregates their outputs.

3.4.1 Agent Topology

The agents are orchestrated as a **LangGraph StateGraph** with the following topology:

```
preprocess_node -> ml_node -> [conditional]
    |-- confidence >= 85% --> agent_a_node
    |-- confidence <  85% --> rag_node --> agent_a_node
                                                |
                                            agent_b_node
                                                |
                                            agent_c_node
                                                |
                                             judge_node
                                                |
                                            output_node --> END
```

Listing 1: LangGraph pipeline topology

Agents execute **sequentially** (A → B → C), not in parallel. Each agent receives the full article text, the ML prediction with confidence, and any retrieved RAG evidence.

3.4.2 Agent Personas

Each agent is implemented as a structured LLM prompt with a distinct role description:

Table 1: Multi-agent persona configuration

Agent	Role	Persona	LLM Provider
Agent A	Conservative	Trusts ML signal and evidence heavily	Groq
Agent B	Sceptical	Challenges evidence quality, highlights uncertainty	Groq
Agent C	Neutral	Balanced, objective reasoning	Groq

All three agents use **Llama 3.3-70B-Versatile** via the Groq API (temperature = 0.3, max tokens = 1,500). Each produces a structured output:

- **Verdict**: `REAL` or `FAKE`
- **Confidence**: 0–100
- **Reasoning**: 3–5 sentence justification

3.5 Final Judge Agent

The judge agent receives all three agent responses, the ML signal, and the agreement statistics (computed before the prompt is assembled). It uses a different LLM—**Llama 3.3-70B-Instruct** via the NVIDIA AI Endpoints API (temperature = 0.2, top-p = 0.7, max tokens = 1,024)—to provide an independent synthesis.

Using a separate LLM provider for the judge reduces the risk of systematic bias from a single model. If the NVIDIA endpoint fails, the system falls back to the Groq API.

The judge produces:

- **Final Verdict**: `REAL` or `FAKE`
- **Final Confidence**: 0–100
- **Consensus Summary**: Synthesis of multi-agent reasoning
- **Dominant Agent**: Which agent was most influential
- **Conflict Resolution**: How disagreements were handled

3.6 Output Assembly and Risk Detection

The final `output_node` assembles the structured response and computes several **risk factors**:

- ML confidence below 70%
- Mixed labels in retrieved RAG evidence

- Weak agent consensus (Low agreement level)
- Insufficient evidence (fewer than 2 retrieved documents)
- Repetitive retrieved content (Jaccard similarity > 0.4 between top-2 documents)

These risk indicators are surfaced in the frontend to help users calibrate their trust in the verdict.

3.7 Agent Workflow

The end-to-end agent workflow follows a structured, multi-stage pipeline orchestrated by LangGraph:

1. **ML Classification** — The cleaned article is vectorised and classified by the Logistic Regression model, producing a prediction (`REAL`/`FAKE`) and a confidence score.
2. **Conditional RAG** — If ML confidence falls below the 85% threshold, the system retrieves the top-5 semantically similar articles from the in-memory ChromaDB store. If confidence is high, this step is skipped to reduce latency.
3. **Multi-Agent Reasoning** — Three agents execute sequentially (A $\rightarrow$ B $\rightarrow$ C). Each agent receives the full article text, the ML prediction with confidence, and any retrieved evidence documents. Despite sequential execution, each agent reasons independently—its prompt contains only the shared evidence, not the outputs of preceding agents—ensuring that each perspective is uncontaminated by the others.
4. **Judge Aggregation** — The judge agent receives all three agent responses together with the ML signal and pre-computed agreement statistics, and synthesises a final verdict.
5. **Output Assembly** — The output node compiles the structured response, computes risk factors, and returns the result to the API caller.

This staged design ensures that each component operates on well-defined inputs and that failures at any node are isolated and recoverable (every node returns fallback values on exception).

3.8 Prompting Strategy

Prompt engineering is central to the quality and consistency of agent outputs. The system uses a shared prompt template with role-specific preambles:

- **Role injection** — Each agent prompt begins with a persona description that shapes reasoning style:
  - Agent A is instructed to trust the ML signal and retrieved evidence heavily.
  - Agent B is instructed to challenge evidence quality and highlight uncertainty.
  - Agent C is instructed to weigh all signals objectively without strong bias.
- **Evidence grounding** — The prompt includes the article text (truncated to 1,500 characters), the ML prediction string, and formatted RAG evidence (label, subject, source, and a 300-character snippet per document).
- **Structured output enforcement** — Agents are required to respond in exactly three labelled fields: `Verdict`, `Confidence`, and `Reasoning`. This deterministic format enables reliable regex-based parsing in the output node.
- **Anti-hallucination directive** — The system message explicitly states: *"Respond ONLY based on the evidence provided. Never invent or hallucinate facts."* This constraint anchors reasoning to the supplied ML and RAG signals.

The judge prompt follows a similar structure but requests additional fields (`Consensus Summary`, `Dominant Agent`, `Conflict Resolution`) and receives the pre-computed agreement level and verdict distribution to aid synthesis.

3.9 State Management

Pipeline state is managed through a **LangGraph `AgentState` TypedDict** that is passed between all nodes. The state dictionary contains the following fields:

Table 2: AgentState fields and their lifecycle

Field	Type	Populated by
`article_text`	`str`	Caller (initial input)
`cleaned_text`	`str`	`preprocess_node`
`ml_prediction`	`str`	`ml_node`
`ml_confidence`	`float`	`ml_node`
`retrieved_docs`	`List[dict]`	`rag_node` (conditional)
`agent_a_response`	`str`	`agent_a_node`
`agent_b_response`	`str`	`agent_b_node`
`agent_c_response`	`str`	`agent_c_node`
`judge_response`	`str`	`judge_node`
`final_report`	`dict`	`output_node`
`error`	`Optional[str]`	Any node (on failure)

Each node receives the full state dictionary and returns a *partial* update containing only the fields it populates. LangGraph merges the returned partial into the running state automatically, ensuring that downstream nodes always see a consistent, progressively enriched view of the data. This merge-based design eliminates manual state-passing logic and guarantees that no node can accidentally overwrite fields it does not own.

4 System Architecture

4.1 Technology Stack

Table 3: Technology stack summary

Layer	**Technology**	**Deployment**
Frontend	React 19, Vite 8, Tailwind CSS 4	Vercel
Backend API	FastAPI, Uvicorn	Render (free tier)
ML Model	scikit-learn (Logistic Regression)	Loaded via joblib
Vector Store	ChromaDB (Ephemeral Client)	In-memory at runtime
Embeddings	all-MiniLM-L6-v2 (precomputed)	Pickle file
Agent Orchestration	LangGraph StateGraph	8-node pipeline
LLM (Agents)	Llama 3.3-70B-Versatile	Groq API
LLM (Judge)	Llama 3.3-70B-Instruct	NVIDIA AI Endpoints
Keep-Alive	GitHub Actions (cron)	Every 14 minutes

5 Results and Analysis

5.1 ML Model Performance

The Logistic Regression classifier was evaluated on a held-out 20% test split of the ISOT dataset.

Table 4: ML model evaluation metrics on the ISOT test set

Metric	**Score**
Accuracy	98.88%
Precision (weighted)	98.88%
Recall (weighted)	98.88%
F1 Score (weighted)	98.88%

Table 5: Per-class performance

Class	Precision	Recall	F1 Score
REAL	0.9835	0.9930	0.9882
FAKE	0.9936	0.9849	0.9892

Table 6: Confusion matrix (test set: 8,980 articles)

	Predicted REAL	Predicted FAKE
Actual REAL	4,240	30
Actual FAKE	71	4,639

The model achieves near-perfect accuracy on the ISOT dataset. However, it is important to note that this performance reflects the dataset's characteristics (clear stylistic differences between the real and fake article sources) and may not generalise to adversarial or out-of-domain content.

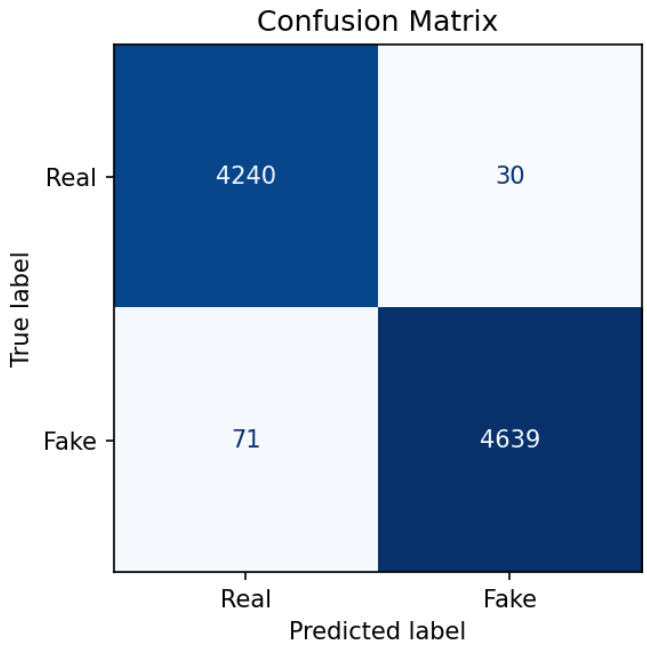


Figure 2: Confusion matrix for the Logistic Regression classifier on the ISOT test set.

5.2 Conditional RAG Behaviour

The 85% confidence threshold was chosen based on empirical observation: articles with ML confidence above this threshold typically receive consistent verdicts from the agents regardless of retrieved evidence. For articles below this threshold, the RAG evidence meaningfully influences agent reasoning by providing ground-truth reference articles for comparison.

5.3 Multi-Agent Agreement Patterns

Qualitative analysis of the system's outputs reveals the following patterns:

- **High agreement (3/3 aligned)**: Observed on clearly fake or clearly real articles where ML confidence is high and RAG evidence is consistent.
- **Medium agreement (2/1 split)**: Typically occurs on articles with ambiguous language or where the ML confidence is moderate (60–85%). Agent B (Sceptical) is most likely to dissent.
- **Low agreement**: Rare; generally indicates genuinely ambiguous content where the system appropriately signals uncertainty.

6 Discussion

6.1 Strengths

1. **Interpretability**: Unlike a black-box classifier, the system provides three independent reasoning perspectives and a synthesis explanation, allowing users to understand *why* a verdict was reached.
2. **Layered confidence**: The combination of ML confidence, agent agreement level, and computed risk factors gives users multiple axes for calibrating trust.
3. **Efficient deployment**: Precomputed embeddings and an ephemeral vector store eliminate the need for persistent storage or GPU inference, enabling deployment on free-tier cloud services.
4. **Conditional RAG**: Skipping retrieval for high-confidence predictions reduces average latency without sacrificing quality on ambiguous inputs.
5. **Dual-provider LLM**: Using separate LLM providers (Groq for agents, NVIDIA for the judge) reduces single-provider bias and adds resilience through fallback.

6.2 Limitations

1. **Dataset bias**: The ISOT dataset contains articles from specific sources (Reuters for real, Politifact-flagged sites for fake). The ML model and RAG evidence may not generalise well to articles from other domains, languages, or writing styles.
2. **Sequential agent execution**: Agents run in series (A $\rightarrow$ B $\rightarrow$ C), adding latency. Parallel execution would reduce response time but requires careful state management.

3. **Cold start**: On Render's free tier, the first request after a sleep period takes 15–30 seconds due to model loading and embedding initialisation.

4. **Single-mode backend**: The frontend offers a "Classical ML" mode, but the backend always runs the full agentic pipeline. A dedicated ML-only endpoint would reduce latency for users who do not need agent reasoning.

5. **Embedding sample size**: Only 5,000 of the approximately 44,000 ISOT articles are embedded for RAG retrieval, limiting evidence coverage.

6. **No real-time fact-checking**: The system analyses linguistic patterns and compares against a static dataset. It does not perform live web searches or consult fact-checking databases.

7 Responsible AI Considerations

The design of this system incorporates several responsible AI principles:

1. **Transparency**: Every verdict is accompanied by per-agent reasoning, evidence references, and risk factors. Users can inspect the full chain of reasoning, not just a final label.

2. **Uncertainty signalling**: When the system is uncertain (low ML confidence, low agent agreement, mixed RAG evidence), risk factors are explicitly surfaced rather than hidden.

3. **Anti-hallucination measures**: LLM agents are prompted with "Respond ONLY based on the evidence provided. Never invent or hallucinate facts." RAG evidence is drawn from a verified dataset.

4. **Multi-perspective reasoning**: By design, the three agents represent different analytical stances, reducing the risk of a single-perspective bias dominating the verdict.

5. **Disclaimer-ready design**: The system is designed as a decision-support tool, not an authority. Risk factors and confidence levels encourage users to exercise their own judgment.

6. **Data privacy**: Article text is processed in real time and not stored. No user data is persisted on the server.

7. **API key security**: Secrets are managed via environment variables and are never committed to the repository. A `.env.example` template documents required variables without exposing real values.

8 Conclusion

This project demonstrates that combining classical machine learning, retrieval-augmented generation, and multi-agent LLM reasoning into a unified pipeline produces a news credibility analysis system that is both accurate and interpretable. The ML layer provides a fast, high-accuracy initial signal; the RAG layer grounds the analysis in verified evidence; and the multi-agent system delivers transparent, multi-perspective reasoning that enables users to understand and evaluate the verdict.

The conditional RAG mechanism and lazy-loading architecture make the system deployable on free-tier cloud infrastructure, demonstrating that sophisticated AI pipelines need not require expensive compute resources.

Future work could extend the system with parallel agent execution for lower latency, integration with live fact-checking APIs for real-time evidence, expansion of the embedding corpus beyond the ISOT dataset, and a dedicated ML-only endpoint for latency-sensitive use cases.

A Frontend Screenshots

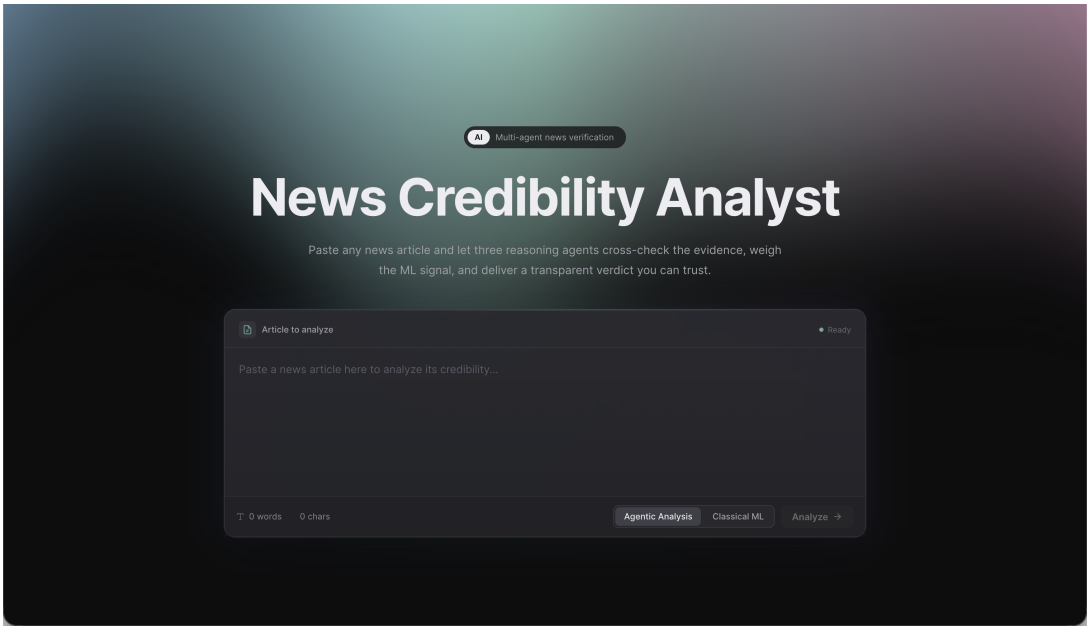


Figure 3: Article input interface with mode selection.

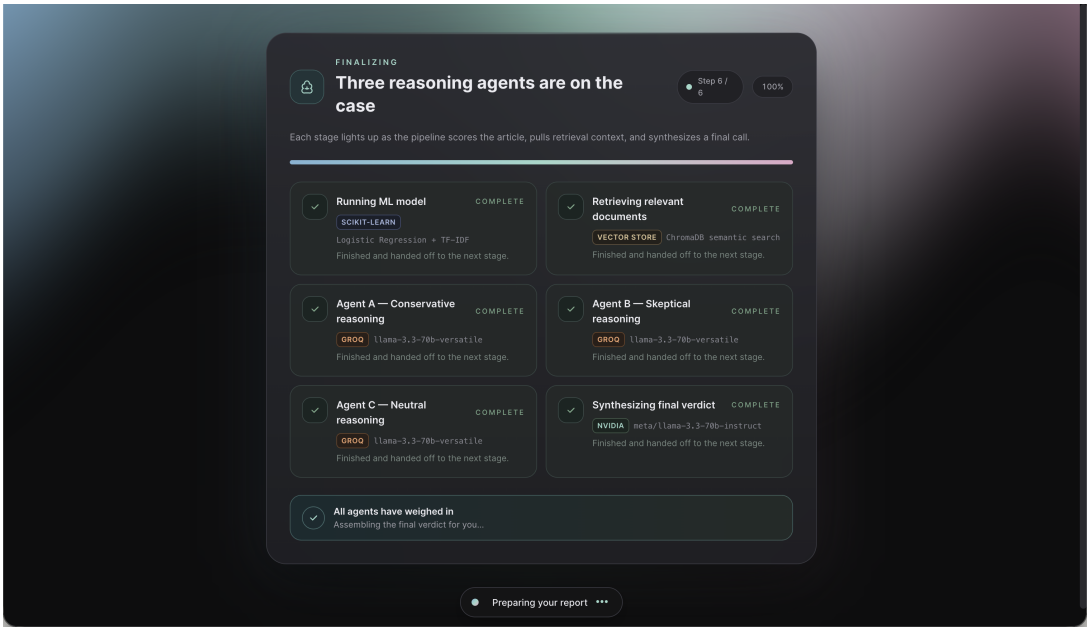


Figure 4: Real-time processing pipeline showing active analysis steps.

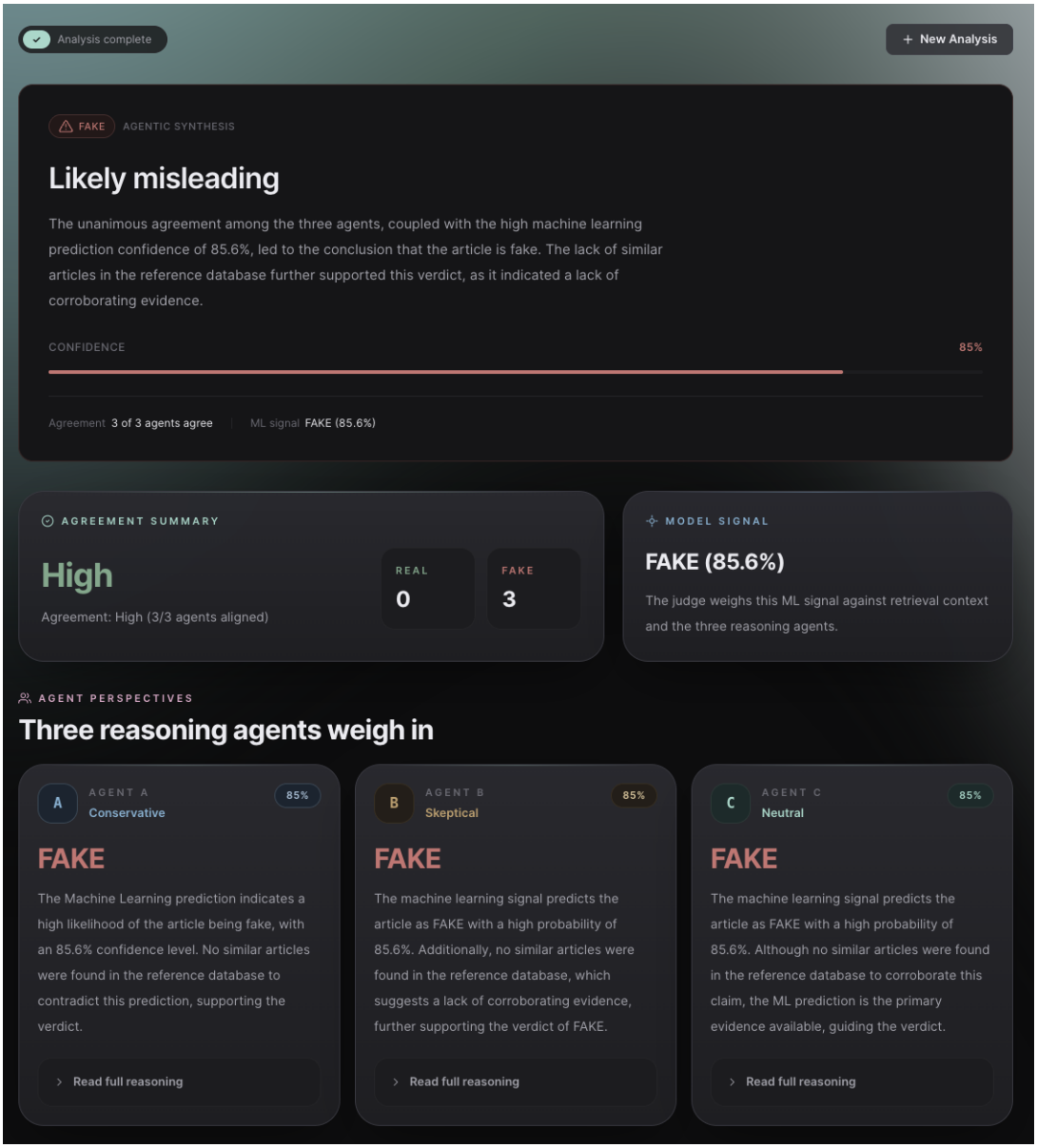


Figure 5: Final verdict display with agent reasoning, evidence summary, and risk indicators.